



Fase 3 Sistema de Domótica Centralizado

Integrantes:

Nombre	Carnet
Marco Rodrigo Funes Bonilla	FB200456
Hazel Alexia García Vásquez	GV252530
Luis Alexander Arteaga Sigüenza	AS230282
Joel Alexander Flores Hernández	FH190646
Gabriel Alejandro Díaz Valencia	DV201086

Arquitectura de Computadoras ACO941 G01T

Ing. Denis Alfredo Altuve Santamaria.

Ingeniería en Ciencias de la Computación.

sábado 30 de mayo del 2026.

Enlaces del proyecto

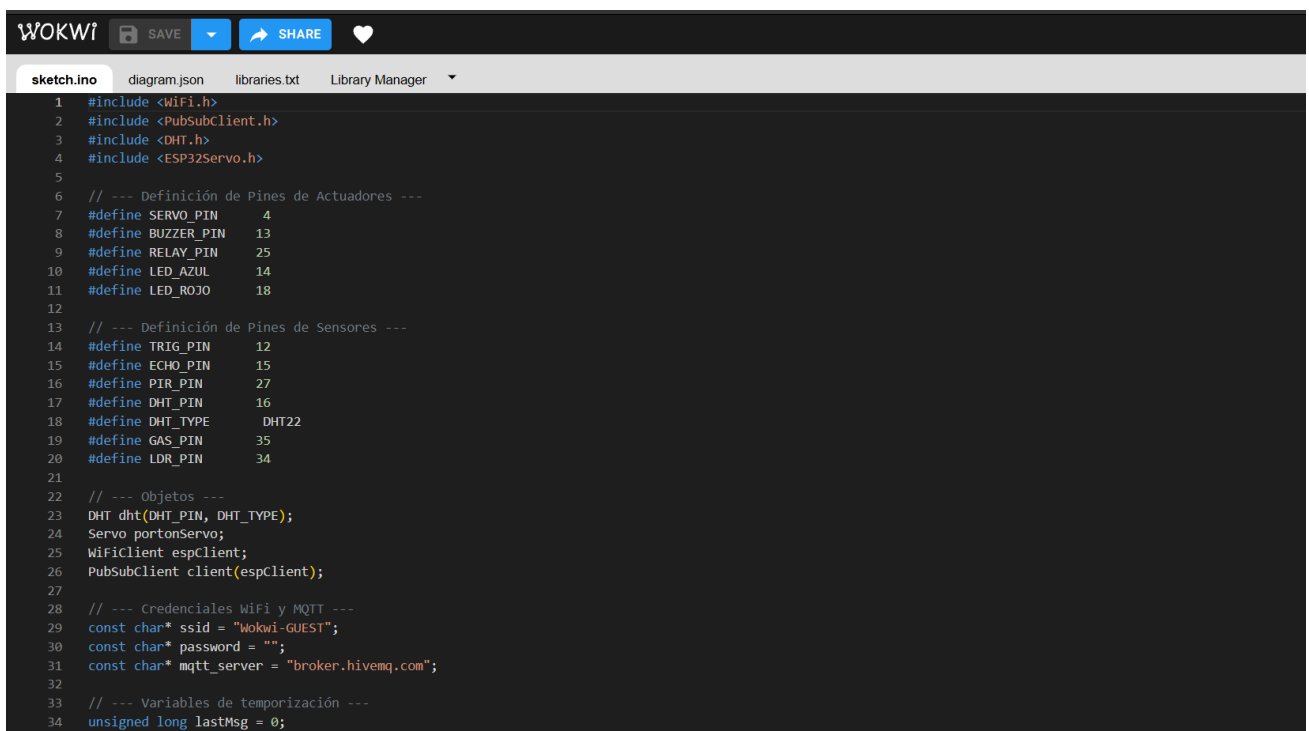
GitHub	https://github.com/usuario/repositorio-domotica
---------------	---

YouTube	<u>FASE 3 ACO</u>
----------------	-----------------------------------

Wokwi (Circuito)	<u>aco Copy (2) - Wokwi ESP32, STM32, Arduino Simulator</u>
-----------------------------	---

Capturas

- Códigos



```
WOKWI [SAVE] [SHARE] [HEART]
sketch.ino diagram.json libraries.txt Library Manager
1 #include <Wifi.h>
2 #include <PubSubClient.h>
3 #include <DHT.h>
4 #include <ESP32Servo.h>
5
6 // --- Definición de Pines de Actuadores ---
7 #define SERVO_PIN 4
8 #define BUZZER_PIN 13
9 #define RELAY_PIN 25
10 #define LED_AZUL 14
11 #define LED_ROJO 18
12
13 // --- Definición de Pines de Sensores ---
14 #define TRIG_PIN 12
15 #define ECHO_PIN 15
16 #define PIR_PIN 27
17 #define DHT_PIN 16
18 #define DHT_TYPE DHT22
19 #define GAS_PIN 35
20 #define LDR_PIN 34
21
22 // --- Objetos ---
23 DHT dht(DHT_PIN, DHT_TYPE);
24 Servo portonServo;
25 WifiClient espClient;
26 PubSubClient client(espClient);
27
28 // --- Credenciales Wifi y MQTT ---
29 const char* ssid = "Wokwi-GUEST";
30 const char* password = "";
31 const char* mqtt_server = "broker.hivemq.com";
32
33 // --- Variables de temporización ---
34 unsigned long lastMsg = 0;
```

```
sketch.ino  diagram.json  libraries.txt  Library Manager  ▼
47 void callback(char* topic, byte* payload, unsigned int length) {
69
70 // --- Control del LED Rojo (alerta) ---
71 if (String(topic) == "udb/domotica/control/alerta") {
72   if (message == "ON") digitalWrite(LED_ROJO, HIGH);
73   else if (message == "OFF") digitalWrite(LED_ROJO, LOW);
74 }
75
76 // --- Control del Buzzer ---
77 if (String(topic) == "udb/domotica/control/buzzer") {
78   if (message == "ON") {
79     digitalWrite(BUZZER_PIN, HIGH);
80     delay(200);
81     digitalWrite(BUZZER_PIN, LOW);
82   }
83 }
84
85 // --- Control del Servo (puerta) ---
86 if (String(topic) == "udb/domotica/control/servo") {
87   if (message == "ABRIR") portonServo.write(90);
88   else if (message == "CERRAR") portonServo.write(0);
89 }
90 }
91
92 void reconnect() {
93   while (!client.connected()) {
94     Serial.print("Intentando conexión MQTT...");
95     String clientId = "ESP32Client-" + String(random(0, 10000));
96     if (client.connect(clientId.c_str())) {
97       Serial.println("Conectado al Broker!");
98       // Suscribirse a TODOS los tópicos de control
99       client.subscribe("udb/domotica/control/ventilador");
100      client.subscribe("udb/domotica/control/luces");
101      client.subscribe("udb/domotica/control/alerta");
102      client.subscribe("udb/domotica/control/buzzer");

```

```
sketch.ino  diagram.json  libraries.txt  Library Manager  ▼
141 void loop() {
148   if (now - lastMsg > 2000) {
149
150     // 2. Lectura de Iluminación - FIX: agregado else para apagar el LED
151     int luzRaw = analogRead(LDR_PIN);
152     client.publish("udb/domotica/sensor/luz", String(luzRaw).c_str());
153     if (luzRaw > 2000) digitalWrite(LED_AZUL, HIGH);
154     else digitalWrite(LED_AZUL, LOW); // FIX: apaga cuando hay luz
155
156     // 3. Sistema de Presencia (Intrusos)
157     int presencia = digitalRead(PIR_PIN);
158     client.publish("udb/domotica/sensor/presencia", String(presencia).c_str());
159     if (presencia == HIGH) {
160       digitalWrite(BUZZER_PIN, HIGH);
161       delay(200);
162       digitalWrite(BUZZER_PIN, LOW);
163     }
164
165     // 4. Control de Acceso (Vehículos)
166     long distancia = leerDistancia();
167     client.publish("udb/domotica/sensor/distancia", String(distancia).c_str());
168     if (distancia < 20 && distancia > 0) {
169       portonServo.write(90);
170     } else {
171       portonServo.write(0);
172     }
173
174     // 5. Monitoreo de Gas Crítico
175     int nivelGas = analogRead(GAS_PIN);
176     client.publish("udb/domotica/sensor/gas", String(nivelGas).c_str());
177     if (nivelGas > 2500) {
178       digitalWrite(LED_ROJO, HIGH);
179     } else {
180       digitalWrite(LED_ROJO, LOW);
181     }
182   }
183 }

```

WOKWI

SAVE

SHARE

sketch.ino

diagram.json

libraries.txt

Library Manager

```
141 void loop() {
142     if (now - lastMsg > 2000) {
143
144         // 3. Sistema de Presencia (Intrusos)
145         int presencia = digitalRead(PIR_PIN);
146         client.publish("udb/domotica/sensor/presencia", String(presencia).c_str());
147         if (presencia == HIGH) {
148             digitalWrite(BUZZER_PIN, HIGH);
149             delay(2000);
150             digitalWrite(BUZZER_PIN, LOW);
151         }
152
153         // 4. Control de Acceso (Vehiculos)
154         long distancia = leerDistancia();
155         client.publish("udb/domotica/sensor/distancia", String(distancia).c_str());
156         if (distancia < 20 && distancia > 0) {
157             portonServo.write(90);
158         } else {
159             portonServo.write(0);
160         }
161
162         // 5. Monitoreo de Gas Critico
163         int nivelGas = analogRead(GAS_PIN);
164         client.publish("udb/domotica/sensor/gas", String(nivelGas).c_str());
165         if (nivelGas > 2500) {
166             digitalWrite(LED_ROJO, HIGH);
167         } else {
168             digitalWrite(LED_ROJO, LOW);
169         }
170
171         Serial.printf("Temp: %.1f°C | Hum: %.1f%% | Luz: %d | Dist: %ldcm | Gas: %d | PIR: %d\n",
172             t, h, luzRaw, distancia, nivelGas, presencia);
173     }
174 }
```

sketch.ino

diagram.json

libraries.txt

Library Manager

```
122 void setup() {
123     // pin config
124
125     pinMode(RELAY_PIN, OUTPUT);
126     pinMode(LED_AZUL, OUTPUT);
127     pinMode(LED_ROJO, OUTPUT);
128     pinMode(BUZZER_PIN, OUTPUT);
129     pinMode(PIR_PIN, INPUT);
130     pinMode(TRIG_PIN, OUTPUT);
131     pinMode(ECHO_PIN, INPUT);
132
133     setup_wifi();
134     client.setServer(mqtt_server, 1883);
135     client.setCallback(callback);
136 }
137
138 void loop() {
139     if (!client.connected()) {
140         reconnect();
141     }
142     client.loop();
143
144     unsigned long now = millis();
145     if (now - lastMsg > 2000) {
146         lastMsg = now;
147
148         // 1. Lectura de Clima
149         float h = dht.readHumidity();
150         float t = dht.readTemperature();
151         if (!isnan(h) && !isnan(t)) {
152             client.publish("udb/domotica/sensor/temperatura", String(t).c_str());
153             client.publish("udb/domotica/sensor/humedad", String(h).c_str());
154             if (t > 30.0) digitalWrite(RELAY_PIN, HIGH);
155             else digitalWrite(RELAY_PIN, LOW); // FIX: apagar relay si baja la temperatura
156         }
157     }
158 }
```

WOKWI

SAVE

SHARE

sketch.ino

diagram.json

libraries.txt

Library Manager

```
35
36
37 void setup_wifi() {
38   delay(10);
39   Serial.println("\nConectando a WiFi...");
40   WiFi.begin(ssid, password);
41   while (WiFi.status() != WL_CONNECTED) {
42     delay(500);
43     Serial.print(".");
44   }
45   Serial.println("\nWiFi Conectado!");
46 }
47
48 void callback(char* topic, byte* payload, unsigned int length) {
49   Serial.print("Mensaje recibido en [");
50   Serial.print(topic);
51   Serial.print("]: ");
52   String message = "";
53   for (int i = 0; i < length; i++) {
54     message += (char)payload[i];
55   }
56   Serial.println(message);
57
58   // --- Control del Relay (ventilador) ---
59   if (String(topic) == "udb/domotica/control/ventilador") {
60     if (message == "ON") digitalWrite(RELAY_PIN, HIGH);
61     else if (message == "OFF") digitalWrite(RELAY_PIN, LOW);
62   }
63
64   // --- Control del LED Azul (luces) ---
65   if (String(topic) == "udb/domotica/control/luces") {
66     if (message == "ON") digitalWrite(LED_AZUL, HIGH);
67     else if (message == "OFF") digitalWrite(LED_AZUL, LOW);
68   }
69 }
```

WOKWI

SAVE

SHARE

sketch.ino

diagram.json

libraries.txt

Library Manager

```
92 void reconnect() {
93   while (!client.connected()) {
94     if (client.connect(clientId.c_str())) {
95       client.subscribe("udb/domotica/control/buzzer");
96       client.subscribe("udb/domotica/control/servo");
97     } else {
98       Serial.print("Falló, rc=");
99       Serial.print(client.state());
100       Serial.println(" Intentando de nuevo en 5 segundos");
101       delay(5000);
102     }
103   }
104 }
105
106 long leerDistancia() {
107   digitalWrite(TRIG_PIN, LOW);
108   delayMicroseconds(2);
109   digitalWrite(TRIG_PIN, HIGH);
110   delayMicroseconds(10);
111   digitalWrite(TRIG_PIN, LOW);
112   return pulseIn(ECHO_PIN, HIGH) * 0.034 / 2;
113 }
114
115 void setup() {
116   Serial.begin(115200);
117   dht.begin();
118   portonServo.attach(SERVO_PIN, 500, 2400);
119   portonServo.write(0);
120
121   pinMode(RELAY_PIN, OUTPUT);
122   pinMode(LED_AZUL, OUTPUT);
123   pinMode(LED_ROJO, OUTPUT);
124   pinMode(BUZZER_PIN, OUTPUT);
125   pinMode(PIR_PIN, INPUT);
126   pinMode(TRIG_PIN, OUTPUT);
127 }
```

WOKWI

SAVE

SHARE

sketch.ino

diagram.json

libraries.txt

Library Manager

```
5  "parts": [
6    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": {} },
7    { "type": "wokwi-servo", "id": "servo1", "top": 74.8, "left": -336, "attrs": {} },
8    { "type": "wokwi-buzzer", "id": "bzi", "top": -74.4, "left": -190.2, "attrs": { "volume": "0.1" } },
9    { "type": "wokwi-relay-module", "id": "relay1", "top": 221, "left": -134.4, "attrs": {} },
10   { "type": "wokwi-led", "id": "led1", "top": -176.4, "left": -121, "attrs": { "color": "blue" } },
11   { "type": "wokwi-resistor", "id": "r1", "top": -176.4, "left": -60, "attrs": { "value": "220" } },
12   { "type": "wokwi-led", "id": "led2", "top": -22.8, "left": 147.8, "attrs": { "color": "red" } },
13   { "type": "wokwi-resistor", "id": "r2", "top": -22.8, "left": 208, "attrs": { "value": "220" } },
14   { "type": "wokwi-hc-sr04", "id": "ultrasonic1", "top": -171.3, "left": -42.5, "attrs": {} },
15   { "type": "wokwi-pir-motion-sensor", "id": "pir1", "top": -159.2, "left": 184.62, "attrs": {} },
16   { "type": "wokwi-dht22", "id": "dht1", "top": 19.5, "left": 263.4, "attrs": {} },
17   { "type": "wokwi-potentiometer", "id": "pot1", "top": 200.3, "left": -269, "attrs": {} },
18   { "type": "wokwi-photoresistor-sensor", "id": "ldr1", "top": 348.8, "left": -47.2, "attrs": {} }
19 ],
20 "connections": [
21   [ "esp:TX", "$serialMonitor:RX", "", [] ],
22   [ "esp:RX", "$serialMonitor:TX", "", [] ],
23   [ "servo1:GND", "esp:GND.1", "black", [ "h0" ] ],
24   [ "servo1:V+", "esp:5V", "green", [ "h0" ] ],
25   [ "servo1:PWM", "esp:4", "green", [ "h0" ] ],
26   [ "bzi:2", "esp:13", "green", [ "v0" ] ],
27   [ "bzi:1", "esp:GND.1", "green", [ "v0" ] ],
28   [ "relay1:VCC", "esp:5V", "red", [ "h0" ] ],
29   [ "relay1:GND", "esp:GND.1", "black", [ "h0" ] ],
30   [ "relay1:IN", "esp:25", "green", [ "h124.8", "v-163.4" ] ],
31   [ "led1:A", "r1:1", "green", [ "v0" ] ],
32   [ "r1:2", "esp:14", "green", [ "v0" ] ],
33   [ "led1:C", "esp:GND.1", "black", [ "v0" ] ],
34   [ "led2:A", "r2:1", "green", [ "v0" ] ],
35   [ "r2:2", "esp:18", "green", [ "v0" ] ],
36   [ "led2:C", "esp:GND.3", "black", [ "v0" ] ],
37   [ "ultrasonic1:VCC", "esp:5V", "red", [ "v0", "h-67.2", "v278.4" ] ],
38   [ "ultrasonic1:TRIG", "esp:12", "green", [ "v9.6", "h-67.6" ] ],
39   [ "ultrasonic1:ECHO", "esp:15", "green", [ "v0", "h95.2", "v249.6" ] ],
40   [ "ultrasonic1:GND", "esp:GND.2", "black", [ "v9.6", "h37.2" ] ],
41   [ "pir1:GND", "esp:GND.2", "black", [ "v0" ] ],
42   [ "pir1:VCC", "esp:5V", "red", [ "v297.6", "h-211.2" ] ],
43   [ "pir1:OUT", "esp:27", "green", [ "v0" ] ],
44   [ "dht1:VCC", "esp:5V", "red", [ "v86.4", "h-249.6" ] ],
45   [ "dht1:SDA", "esp:16", "green", [ "v0" ] ],
46   [ "dht1:GND", "esp:GND.2", "black", [ "v19.2", "h-105.6", "v-144" ] ],
47   [ "pot1:GND", "esp:GND.2", "black", [ "v38.4", "h364.8", "v-268.8" ] ],
48   [ "pot1:VCC", "esp:5V", "red", [ "v9.6", "h66.4", "v-76.8" ] ],
49   [ "pot1:SIG", "esp:35", "green", [ "v19.2", "h86", "v-201.6" ] ],
50   [ "ldr1:VCC", "esp:5V", "red", [ "h9.6", "v-96", "h-124.8" ] ],
51   [ "ldr1:GND", "esp:GND.2", "black", [ "h67.2", "v-336.4" ] ],
52   [ "ldr1:AO", "esp:34", "green", [ "h38.4", "v-144.7", "h-115.2" ] ]
53 ],
54 "dependencies": {}
55 }
```

WOKWI

SAVE

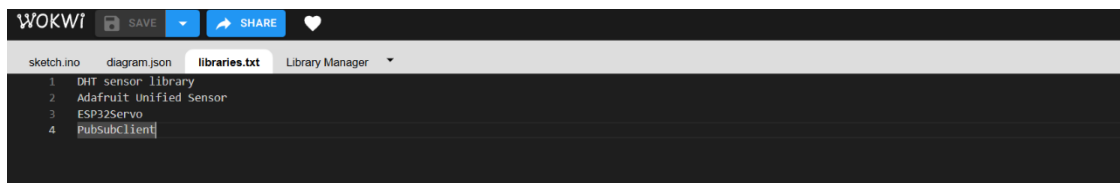
SHARE

sketch.ino

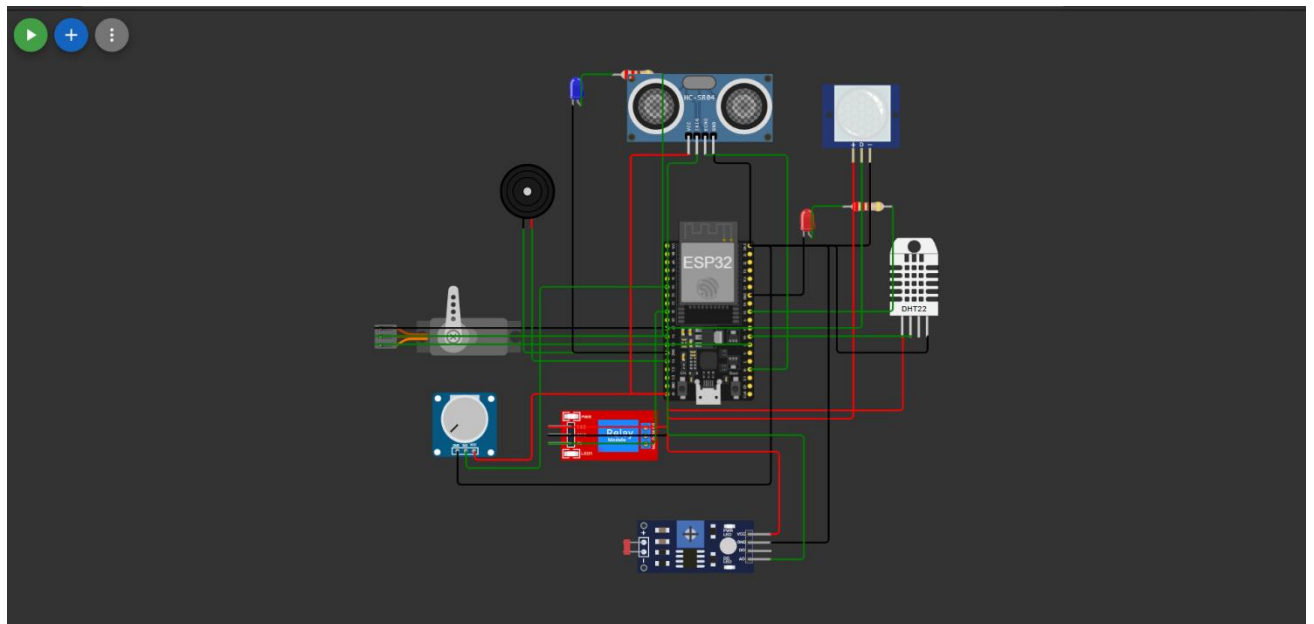
diagram.json

libraries.txt

Library Manager



- Dashboard



- Porcentaje Alcanzado

Módulo	Estado	%
Integración de hardware y programación	✓ Completo	100%
Comunicación MQTT	✓ Completo	100%
Dashboard de monitoreo y control	✓ Completo	100%
Simulación funcionando	✓ Completo	100%
Evidencias y repositorio Git	✓ Completo	100%
Presentación final	✗ Pendiente	—